

```
In [52]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import math
```

```
In [2]: df = pd.read_csv("Real_estate.csv")
df.head()
```

```
Out[2]:
```

	ID	Has_Garden	Num_Bathrooms	Num_Bedrooms	Square_Feet	Year_Built	Price
0	0	0	3	3	282.424413	2015	787131.569691
1	1	1	3	2	290.611824	1929	620457.367069
2	2	1	1	3	208.773413	1990	657717.309665
3	3	0	3	1	261.218828	1962	539872.127390
4	4	0	2	3	61.612603	1914	428376.720807

```
In [3]: df.shape
```

```
Out[3]: (500, 7)
```

```
In [4]: df.sample(5)
```

Out[4]:

	ID	Has_Garden	Num_Bathrooms	Num_Bedrooms	Square_Feet	Year_Built	Price
425	425	0	1	2	208.076458	2002	554247.365996
103	103	1	2	1	116.300592	1920	299387.116387
369	369	1	1	1	177.835600	1917	455326.610110
56	56	1	1	5	173.794228	1901	524483.184244
10	10	1	3	1	94.359886	2003	503344.391660

In [5]: `df.nunique()`

Out[5]:

ID	500
Has_Garden	2
Num_Bathrooms	3
Num_Bedrooms	5
Square_Feet	500
Year_Built	121
Price	500

dtype: int64

In [6]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   ID              500 non-null   int64
1   Has_Garden      500 non-null   int64
2   Num_Bathrooms   500 non-null   int64
3   Num_Bedrooms    500 non-null   int64
4   Square_Feet     500 non-null   float64
5   Year_Built      500 non-null   int64
6   Price           500 non-null   float64
dtypes: float64(2), int64(5)
memory usage: 27.5 KB
```

```
In [7]: df.describe()
```

```
Out[7]:
```

	ID	Has_Garden	Num_Bathrooms	Num_Bedrooms	Square_Feet	Year_Built	Price
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	249.500000	0.536000	1.976000	2.958000	174.640428	1957.604000	582209.629529
std	144.481833	0.499202	0.820225	1.440968	74.672102	35.491781	122273.390345
min	0.000000	0.000000	1.000000	1.000000	51.265396	1900.000000	276892.470136
25%	124.750000	0.000000	1.000000	2.000000	110.319923	1926.000000	503080.344140
50%	249.500000	1.000000	2.000000	3.000000	178.290937	1959.000000	574724.113347
75%	374.250000	1.000000	3.000000	4.000000	239.031220	1988.000000	665942.301274
max	499.000000	1.000000	3.000000	5.000000	298.241199	2022.000000	960678.274291

```
In [8]: df.isnull().sum()
```

```
Out[8]: ID          0
Has_Garden        0
Num_Bathrooms     0
Num_Bedrooms      0
Square_Feet       0
Year_Built        0
Price            0
dtype: int64
```

```
In [9]: df["Price"] = df["Price"].round(2)
df.head()
```

Out[9]:

	ID	Has_Garden	Num_Bathrooms	Num_Bedrooms	Square_Feet	Year_Built	Price
0	0	0	3	3	282.424413	2015	787131.57
1	1	1	3	2	290.611824	1929	620457.37
2	2	1	1	3	208.773413	1990	657717.31
3	3	0	3	1	261.218828	1962	539872.13
4	4	0	2	3	61.612603	1914	428376.72

In [10]: `df.duplicated().sum()`

Out[10]: `np.int64(0)`

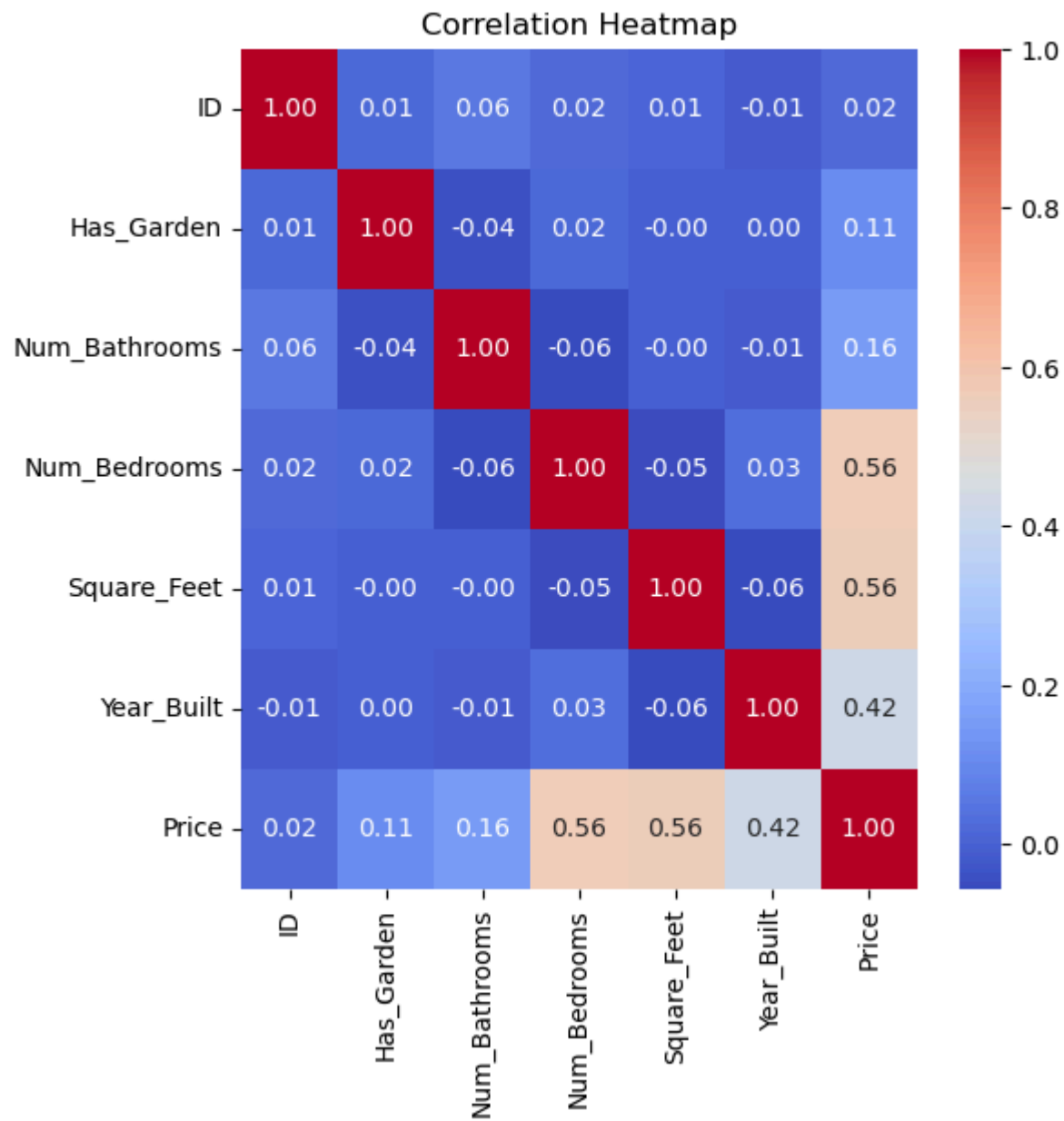
In [11]: `df.corr()`

Out[11]:

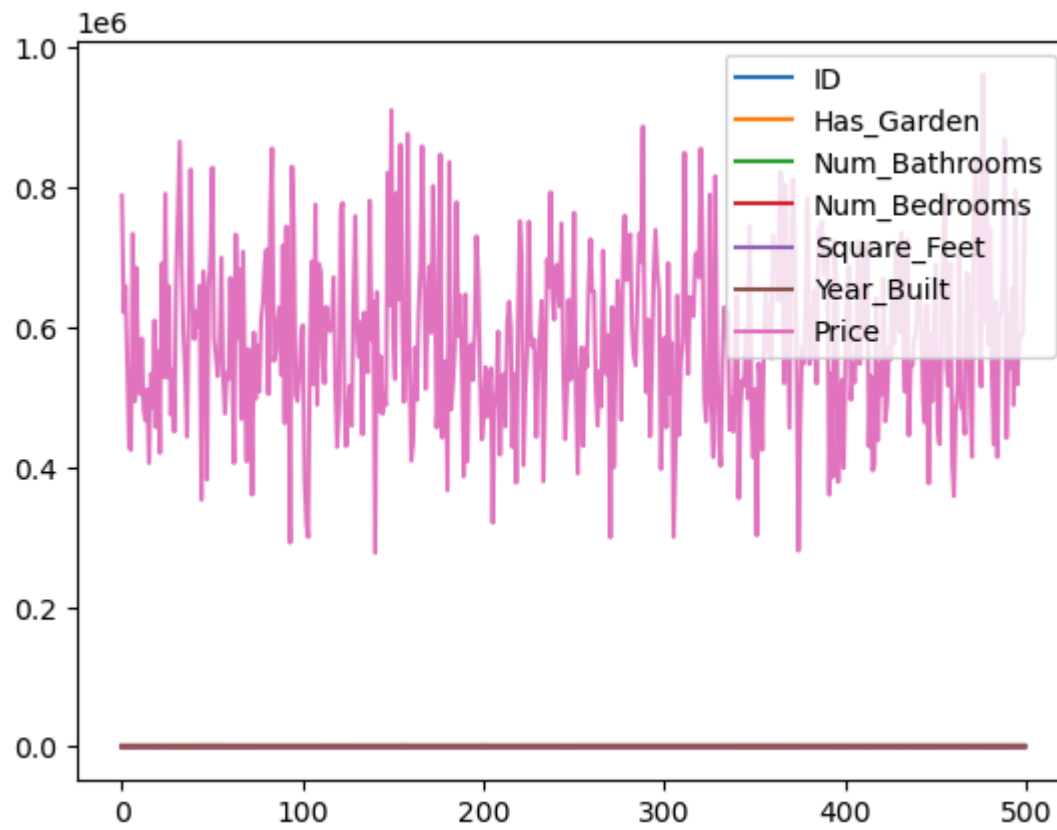
	ID	Has_Garden	Num_Bathrooms	Num_Bedrooms	Square_Feet	Year_Built	Price
ID	1.000000	0.013892	0.055567	0.022173	0.010443	-0.013426	0.020695
Has_Garden	0.013892	1.000000	-0.037040	0.017429	-0.000978	0.000693	0.109196
Num_Bathrooms	0.055567	-0.037040	1.000000	-0.056808	-0.001667	-0.013407	0.156689
Num_Bedrooms	0.022173	0.017429	-0.056808	1.000000	-0.049918	0.031806	0.563973
Square_Feet	0.010443	-0.000978	-0.001667	-0.049918	1.000000	-0.055062	0.558604
Year_Built	-0.013426	0.000693	-0.013407	0.031806	-0.055062	1.000000	0.418293
Price	0.020695	0.109196	0.156689	0.563973	0.558604	0.418293	1.000000

In [12]:

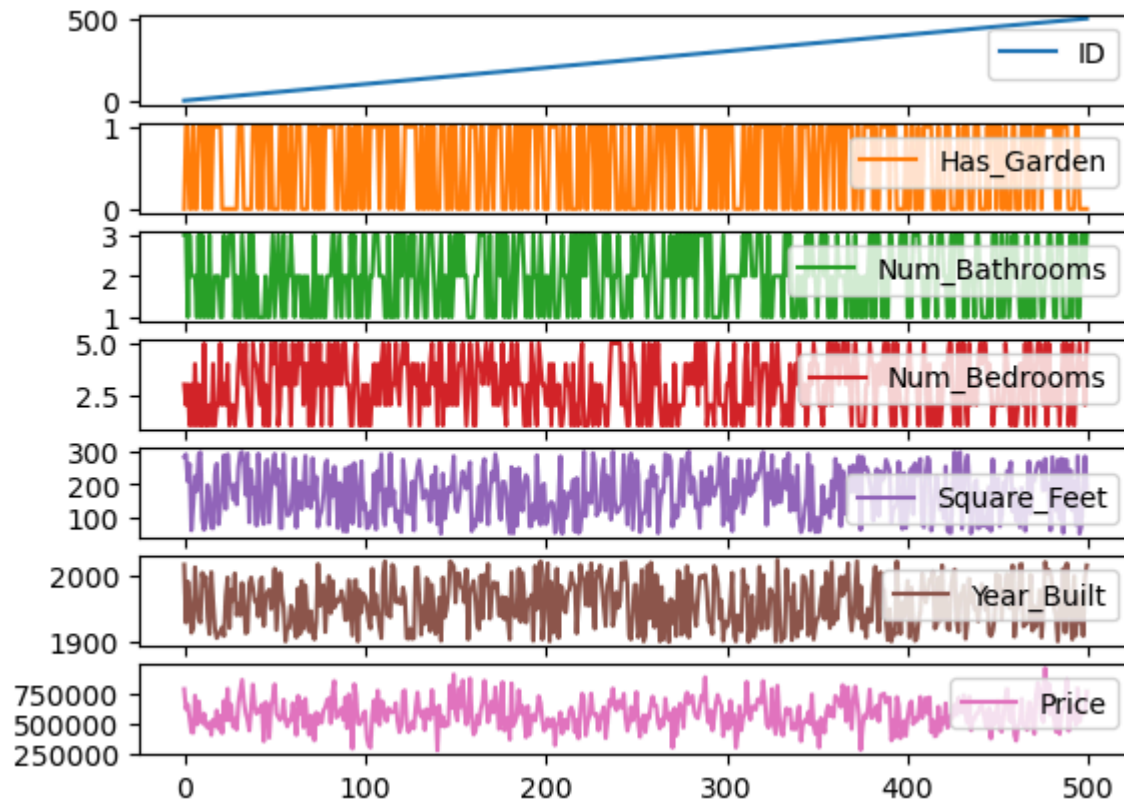
```
plt.figure(figsize=(6, 6))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



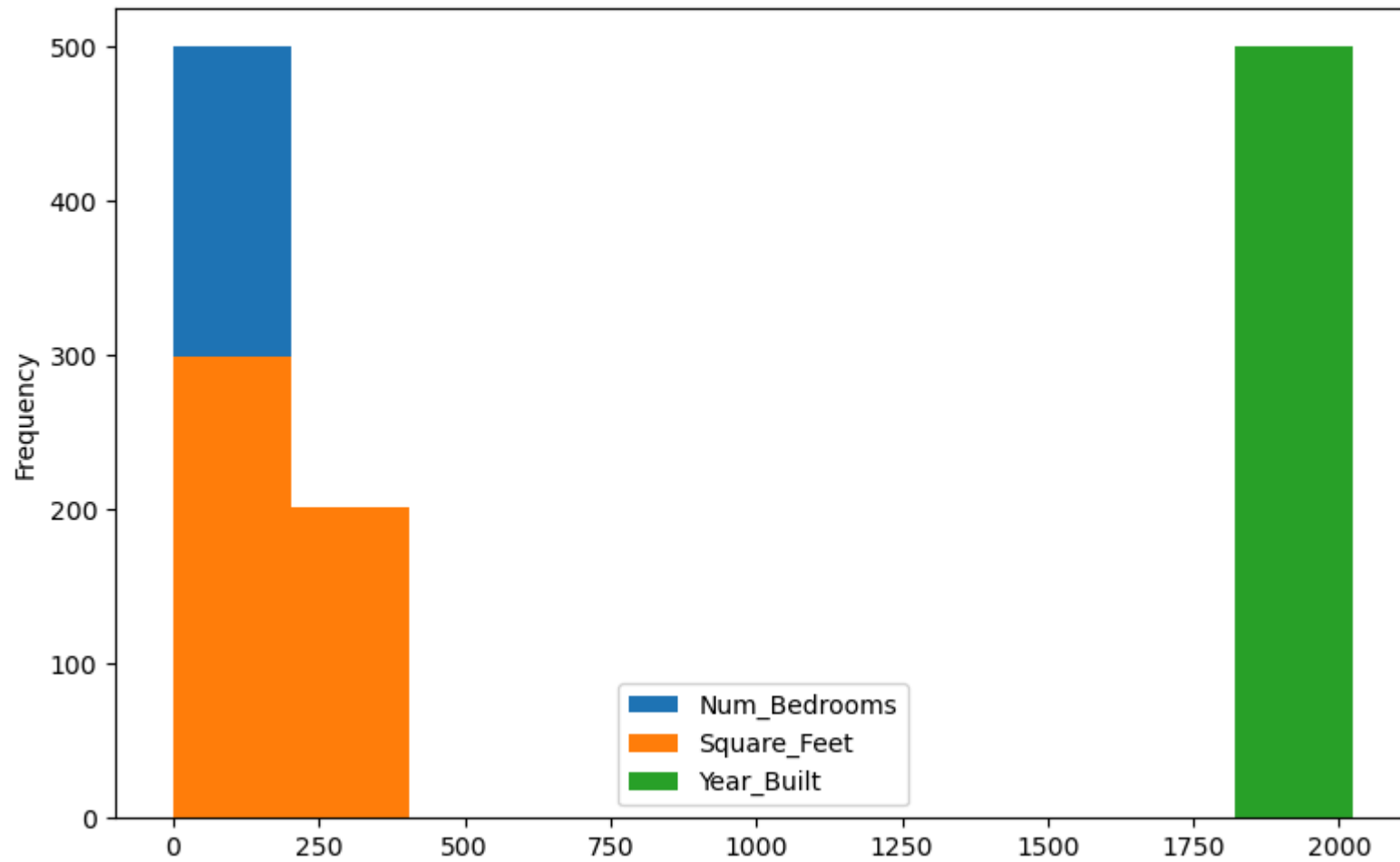
```
In [13]: df.plot(kind = "line")  
plt.show()
```



```
In [14]: df.plot(kind="line", subplots=True)
plt.show()
```

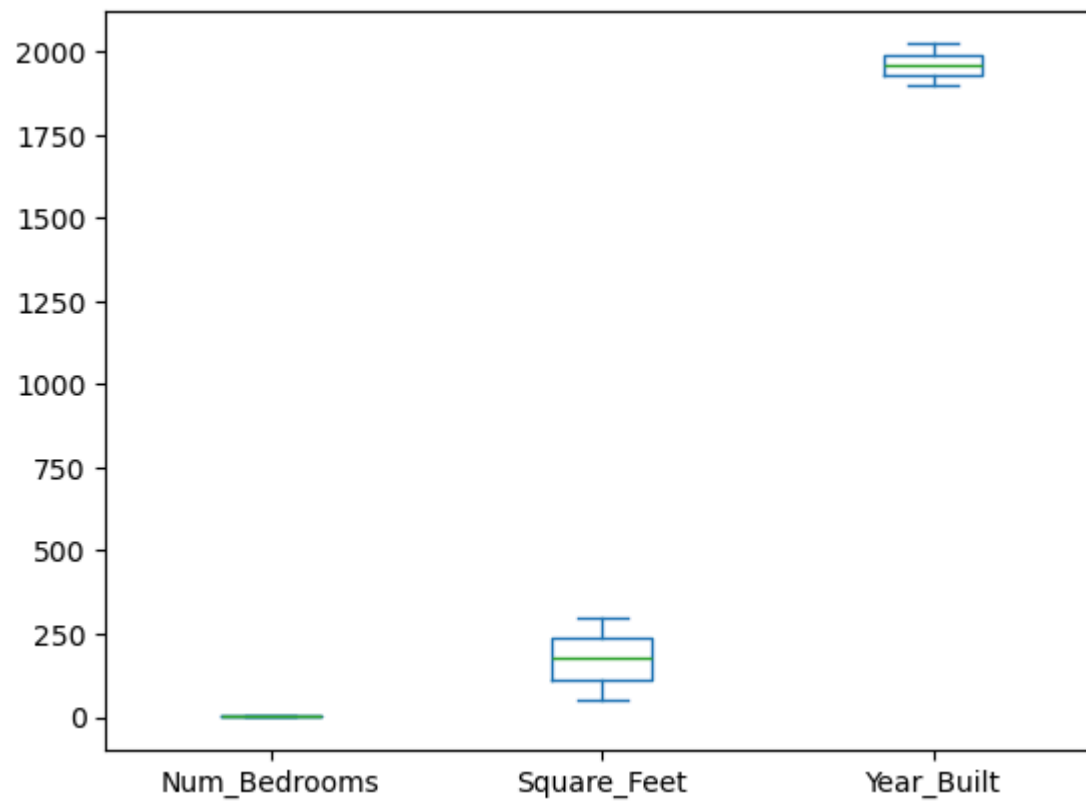


```
In [15]: df[['Num_Bedrooms', 'Square_Feet', 'Year_Built']].plot.hist(figsize=(8,5))  
plt.tight_layout()  
plt.show()
```



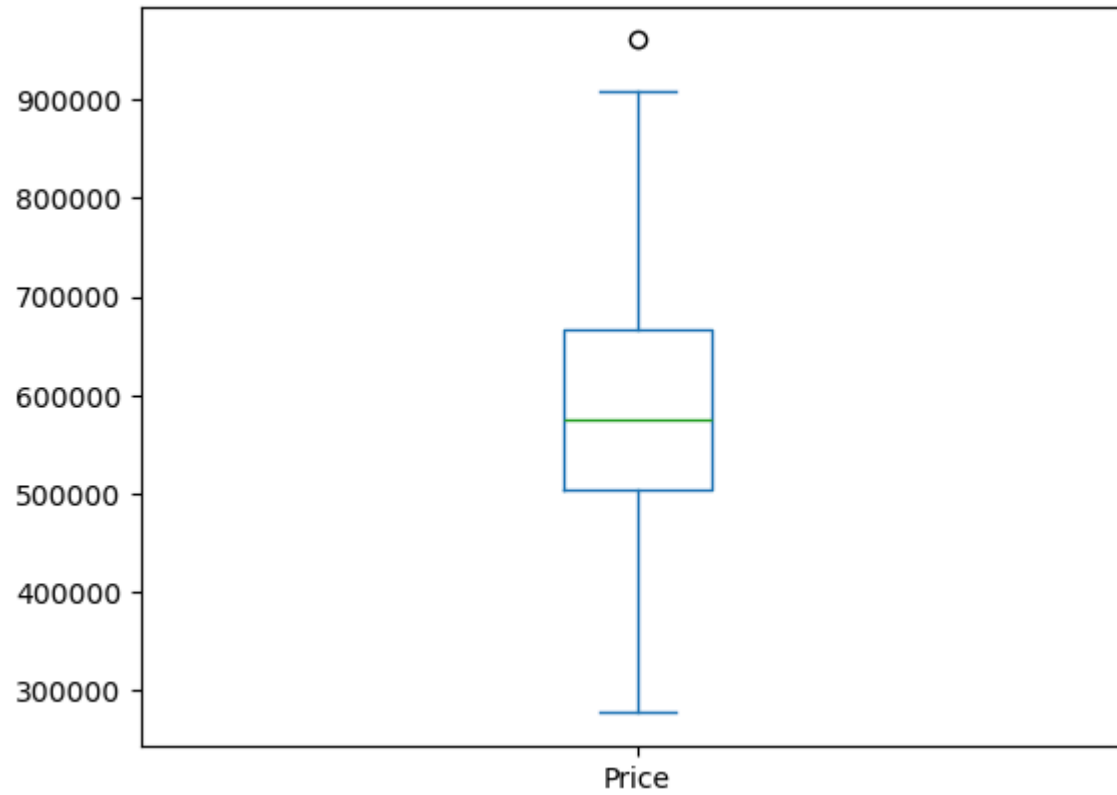
```
In [16]: df[['Num_Bedrooms', 'Square_Feet', 'Year_Built']].plot.box()
```

```
Out[16]: <Axes: >
```

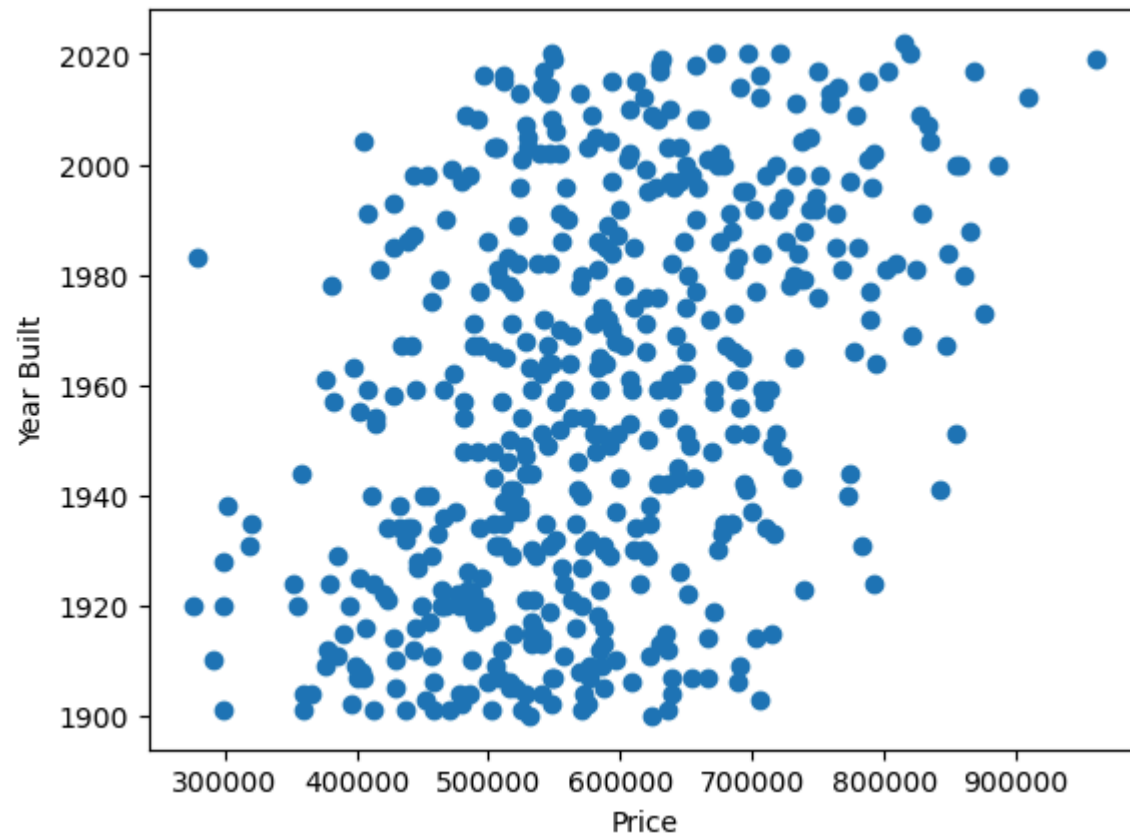



```
In [17]: df["Price"].plot.box()
```

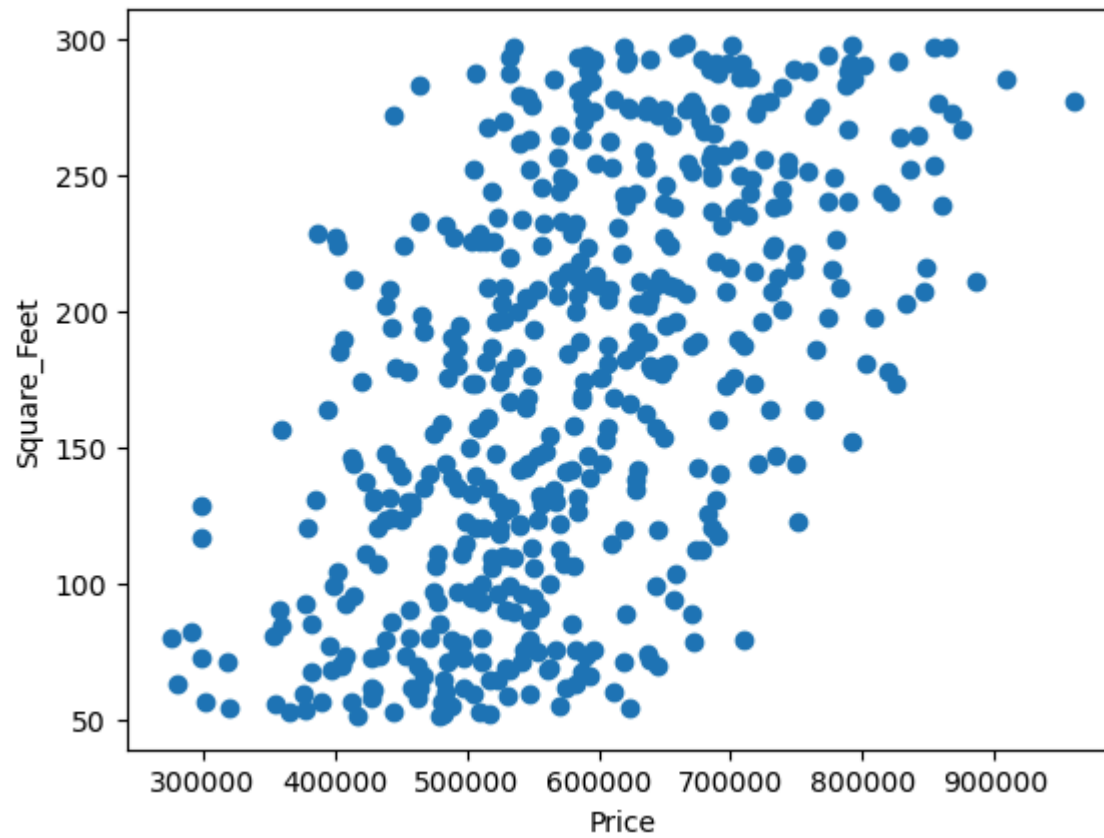
```
Out[17]: <Axes: >
```



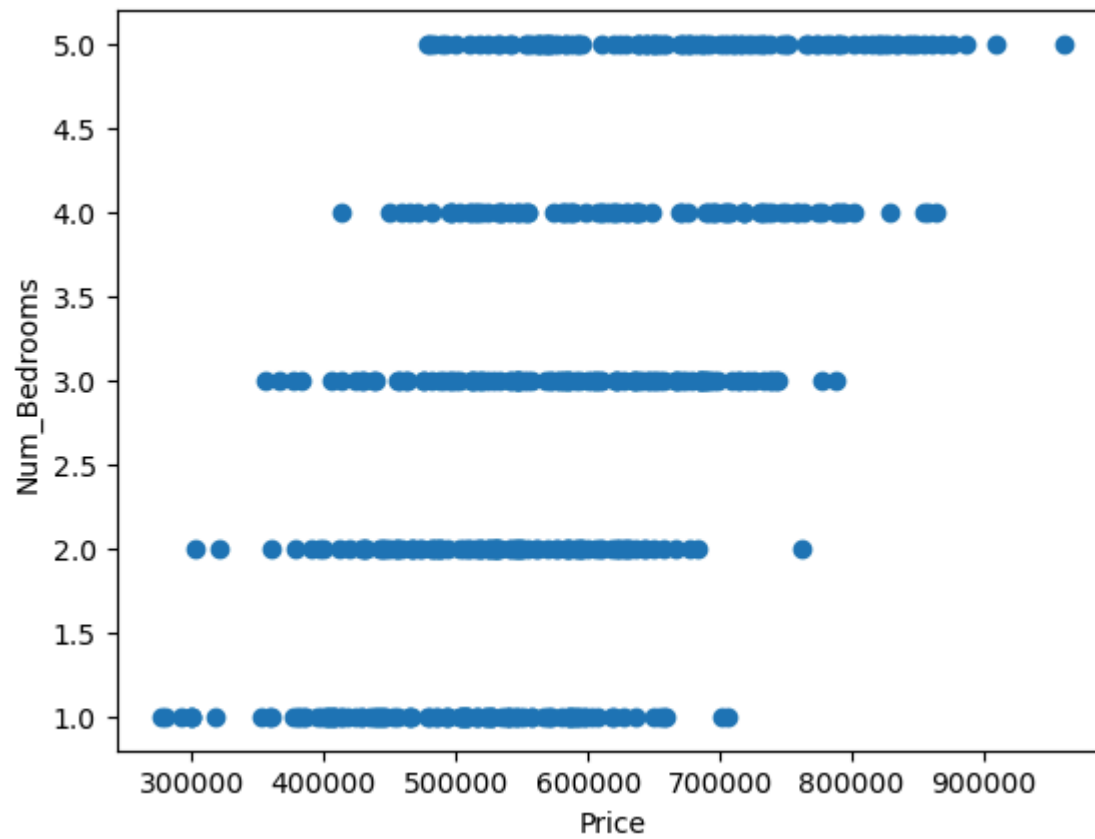
```
In [18]: plt.scatter(x=df["Price"], y=df["Year_Built"])  
plt.xlabel("Price")  
plt.ylabel("Year Built")  
plt.show()
```



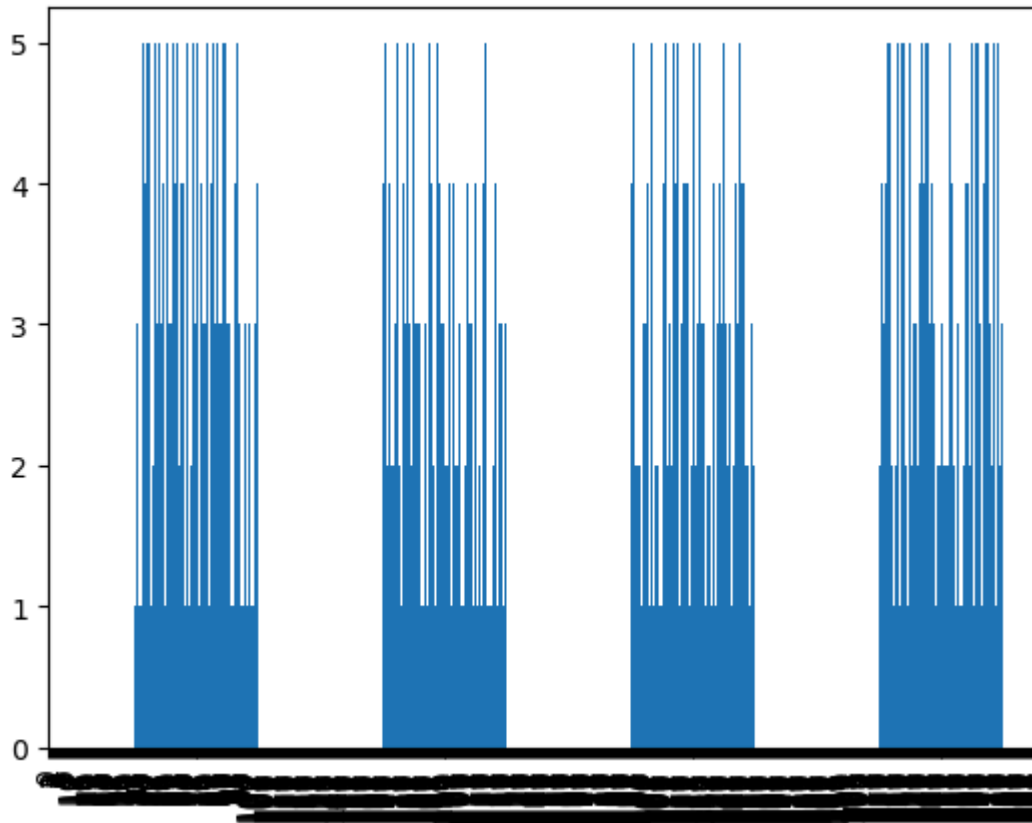
```
In [19]: plt.scatter(x=df["Price"], y=df["Square_Feet"])  
plt.xlabel("Price")  
plt.ylabel("Square_Feet")  
plt.show()
```



```
In [20]: plt.scatter(x=df["Price"], y=df["Num_Bedrooms"])  
plt.xlabel("Price")  
plt.ylabel("Num_Bedrooms")  
plt.show()
```



```
In [21]: df['Num_Bedrooms'].plot.bar()  
plt.show()
```



```
In [24]: X = df.drop(['Price'],axis = 1)
        y = df['Price']
```

```
In [26]: X_train , X_test , y_train , y_test = train_test_split(X ,y, test_size = 0.2)
```

```
In [27]: train_data = X_train.join(y_train)
```

```
In [28]: train_data
```

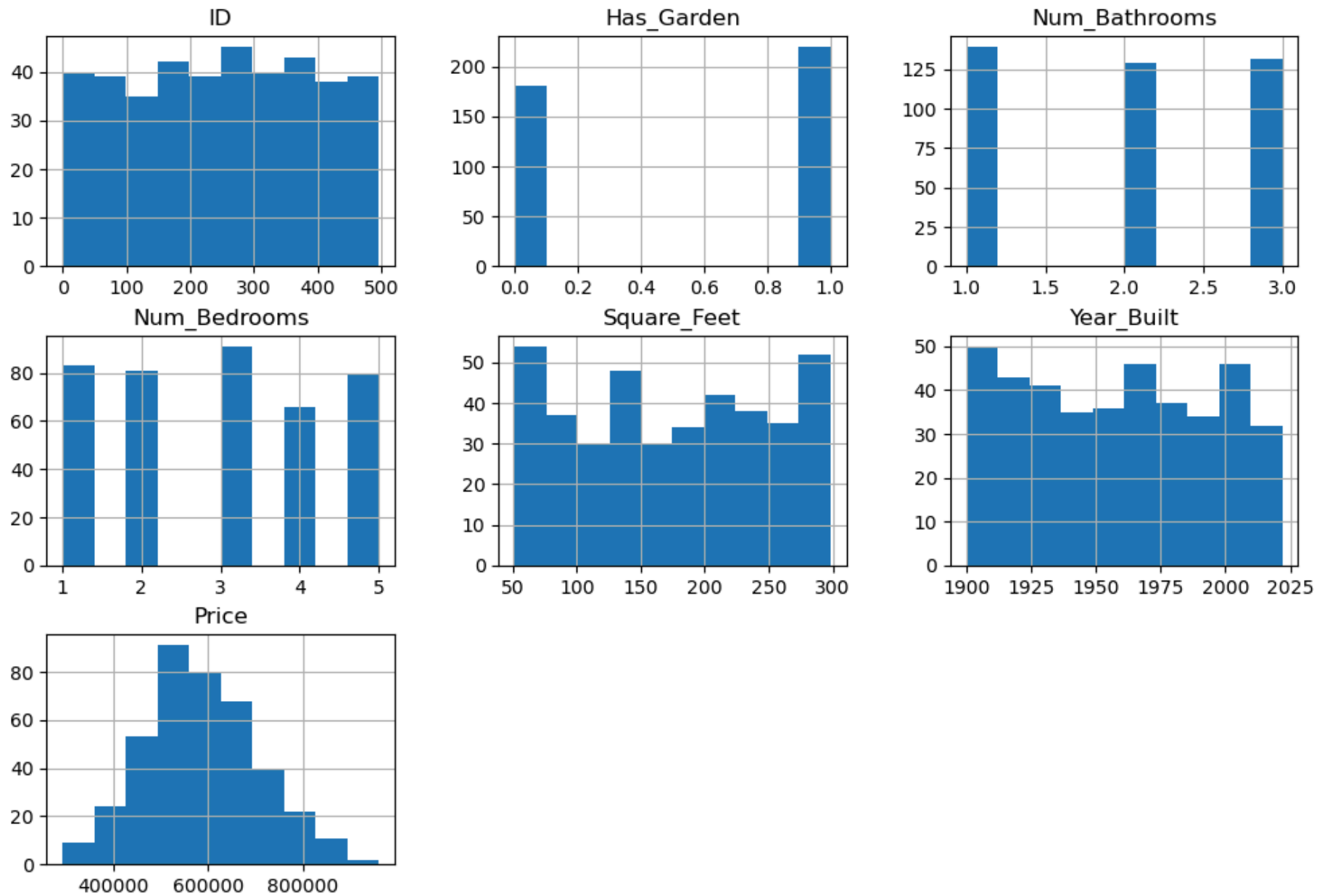
Out[28]:

	ID	Has_Garden	Num_Bathrooms	Num_Bedrooms	Square_Feet	Year_Built	Price
217	217	0	3	2	219.391090	1913	532504.76
258	258	1	3	1	238.093632	2018	657042.15
283	283	0	1	5	129.230501	1924	557099.51
44	44	1	1	1	80.509559	1924	352899.27
352	352	1	3	2	86.223718	1964	547733.42
...	...	...	...	...	...	...	...
266	266	1	2	3	290.805635	1957	708362.29
339	339	0	2	3	256.614367	1908	568484.72
205	205	1	1	2	54.146957	1935	320014.71
216	216	0	3	1	57.857296	1985	427818.45
75	75	0	1	5	106.983791	1954	573486.15

400 rows × 7 columns

In [33]: `train_data.hist(figsize=(12,8))`

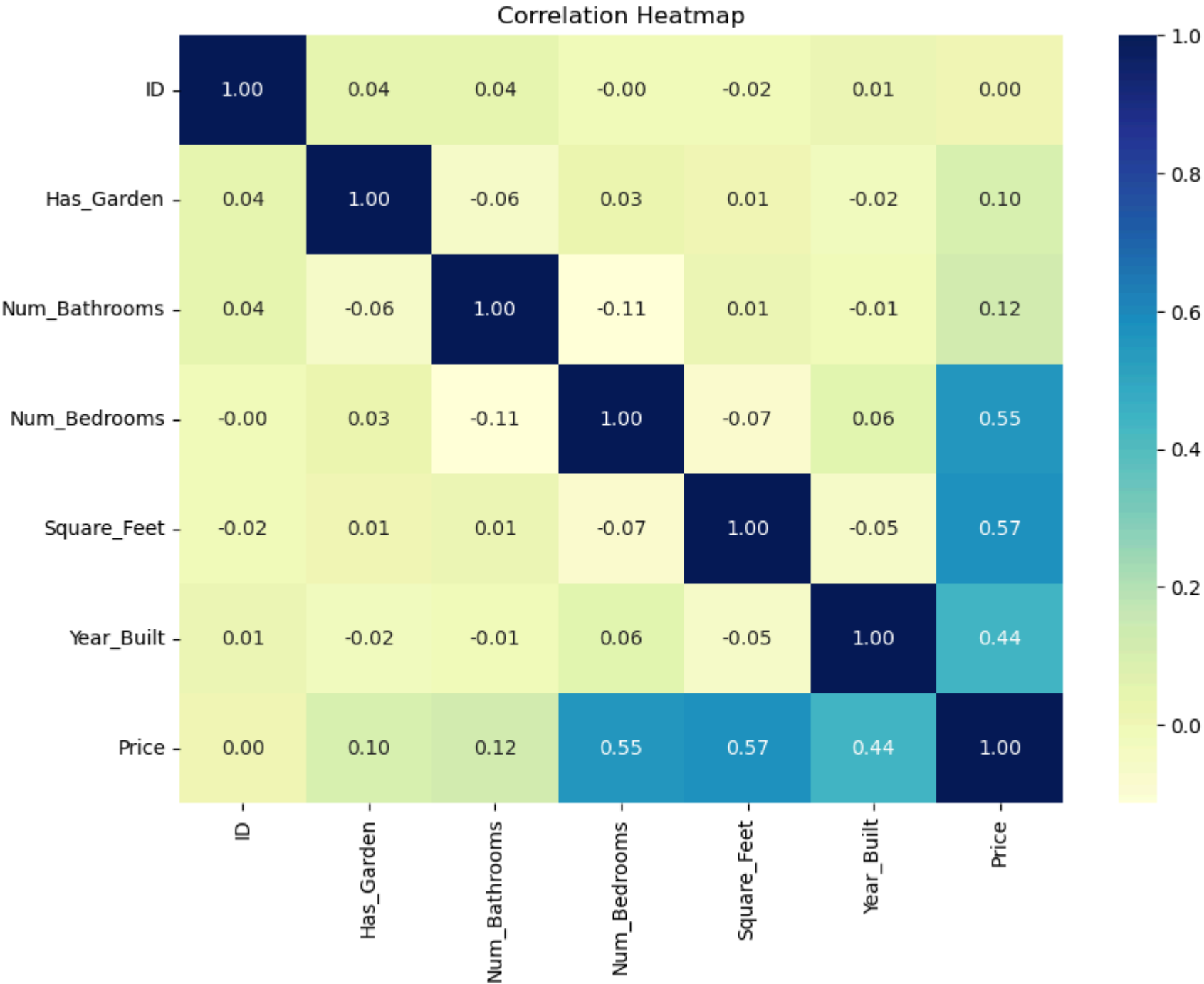
Out[33]: array([[<Axes: title={'center': 'ID'}>,
 <Axes: title={'center': 'Has_Garden'}>,
 <Axes: title={'center': 'Num_Bathrooms'}>],
 [<Axes: title={'center': 'Num_Bedrooms'}>,
 <Axes: title={'center': 'Square_Feet'}>,
 <Axes: title={'center': 'Year_Built'}>],
 [<Axes: title={'center': 'Price'}>, <Axes: >, <Axes: >]],
 dtype=object)



```
In [41]: plt.figure(figsize=(10, 7))
sns.heatmap(train_data.corr(), annot=True, fmt=".2f", cmap="YlGnBu")
```



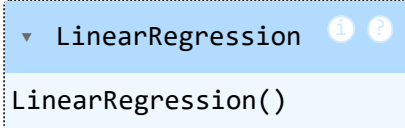
```
plt.title("Correlation Heatmap")  
plt.show()
```



```
In [43]: X_train = train_data.drop(['Price'], axis = 1)
y_train = train_data['Price']
```

```
reg = LinearRegression()
```

```
In [44]: reg.fit(X_train, y_train)
```

```
Out[44]: 
LinearRegression()
```

```
In [45]: reg.score(X_test, y_test)
```

```
Out[45]: 0.8746246678697556
```

```
In [47]: y_pred = reg.predict(X_test)
```

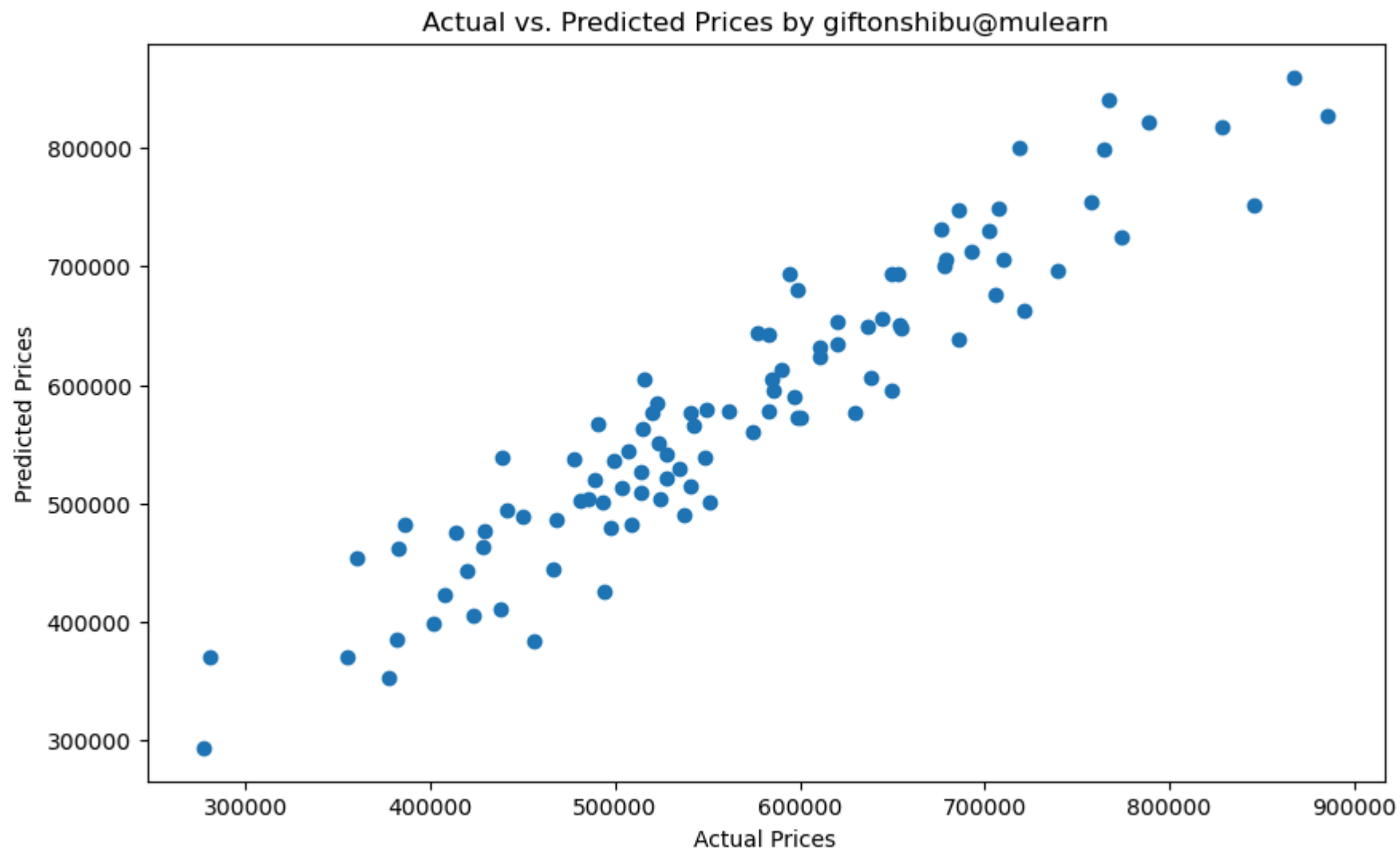
```
In [50]: mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
In [56]: print(f"Root Mean Squared Error: {math.sqrt(mse)}")
print(f"R-squared: {r2}")
```

Root Mean Squared Error: 44632.60747731038

R-squared: 0.8746246678697556

```
In [64]: plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs. Predicted Prices by giftonshibu@mulearn")
plt.savefig('actual_vs_predicted.png')
```



```
In [67]: # Create a DataFrame with actual and predicted values
results = pd.DataFrame({
    "Actual_Price": y_test.values,
    "Predicted_Price": y_pred
})

# Save to CSV
```

```
results.to_csv("predictions_vs_actual.csv", index=False)  
results.head(10)
```

Out[67]:

	Actual_Price	Predicted_Price
0	402001.14	398394.068336
1	676685.28	731133.405078
2	516038.13	605357.264606
3	677859.95	700575.630855
4	584792.07	604783.380101
5	428668.28	463378.819486
6	549076.81	578949.623206
7	654130.38	650403.081180
8	497402.48	479552.518748
9	423506.53	405833.489723

In []: